

Laboratory report of Nanjing University of Information Science and Technology Experiment

Title	Lab 6	Date	2025.5.16	Class	24AC
Name	ZhuangZhong	ID	202483910032		

1. Objectives

The main objective of this lab is to understand and practice fundamental Knative operations for building, deploying, and managing serverless cloud-native applications on Kubernetes, including:

Installing Knative Serving component on an existing Istio-enabled Kubernetes cluster

Installing Knative Eventing component for event-driven architecture

Setting up Minikube with adequate resources for Knative (15GB memory, 8 CPUs)

Creating and deploying a Knative Service (hello-world)

Configuring Istio ingress gateway for external access to Knative services

Understanding Knative Functions (func) CLI for function development

Understanding Knative Eventing with PingSource and Trigger mechanisms

Monitoring Knative components using kubectl

2. Environment Preparation

Knative was installed on Windows 11 using Docker Desktop with Kubernetes (v1.34.0). Minikube was configured with specific resource requirements for Knative (15GB memory, 8 CPUs). Istio was already installed from Lab 4 and serves as the networking layer for Knative.

2.1 Minikube Cluster Configuration

A new Minikube cluster was started with the following command to ensure sufficient resources for Knative components:

```
bash
```

```
minikube start --driver=docker --memory=15360 --cpus=8
```

```
--base-image=registry.k8s.io/kicbase/stable:v0.0.48 --kubernetes-version=v1.34.0
```

This configuration allocates 15GB RAM and 8 CPU cores, which is necessary for running Knative Serving, Knative Eventing, Istio, and the function runtime simultaneously.

2.2 Verifying Tools Installation

The kn CLI (Knative client) version was verified to ensure compatibility with Knative v1.17.0. The func CLI (Knative Functions) was also installed for serverless function development.

```

C:\Users\联想\istio-1.29.2>move kn.exe C:\Windows\System32\
Access is denied.
0 file(s) moved.

C:\Users\联想\istio-1.29.2>kn version
Version: v1.17.0
Build Date: 2025-01-22 16:25:15
Git Revision: f7347245
Supported APIs:
* Serving
  - serving.knative.dev/v1 (knative-serving v1.17.0)
* Eventing
  - sources.knative.dev/v1 (knative-eventing v1.17.0)
  - eventing.knative.dev/v1 (knative-eventing v1.17.0)

C:\Users\联想\istio-1.29.2>ren func_windows_amd64.exe func.exe

C:\Users\联想\istio-1.29.2>move func.exe C:\Windows\System32\
Access is denied.
0 file(s) moved.

C:\Users\联想\istio-1.29.2>func version
v0.44.0

C:\Users\联想\istio-1.29.2>

```

The output shows:

kn version: v1.17.0

Build Date: 2025-01-22

Supported APIs: Serving (v1) and Eventing (v1)

The func CLI was renamed from func_windows_amd64.exe to func.exe and version v0.44.0 was confirmed.

3. Experiment Implementation and Results

3.1 Installing Knative Serving

Knative Serving was installed using YAML manifests. The installation consists of two main components:

Serving CRDs (Custom Resource Definitions)

Serving Core (controller, webhook, autoscaler, activator)

After applying both YAML files, the pods in the knative-serving namespace were verified.

```

C:\Users\联想\istio-1.29.2>kubectl get pods -n knative-serving
NAME                                READY   STATUS    RESTARTS   AGE
activator-595d9dccc8-8mmbw          1/1     Running   0           8m29s
autoscaler-6bffb656cd-hw5v4         1/1     Running   0           8m28s
controller-6b75568fc-k49gf          1/1     Running   0           8m28s
net-istio-controller-5747656bdd-kd5rl 1/1     Running   0           6m20s
net-istio-webhook-645496f8b9-24nxc    1/1     Running   0           6m20s
webhook-76bcc8bb77-99qhd            1/1     Running   0           8m28s

C:\Users\联想\istio-1.29.2>kubectl get pods -n knative-eventing
NAME                                READY   STATUS              RESTARTS   AGE
eventing-controller-86f855d85-mx6v7 0/1     ImagePullBackOff    0           35s
eventing-webhook-74765bc895-9bm8j     0/1     ContainerCreating   0           34s
imc-controller-fbd87f5ff-7gv7t         0/1     ContainerCreating   0           33s
imc-dispatcher-5c6c6ddd98-cgzld        0/1     ContainerCreating   0           33s
job-sink-7c69f84bb5-mbvsl             1/1     Running             0           35s
mt-broker-controller-55f7cdbcc7-xrbc9 0/1     ContainerCreating   0           31s
mt-broker-filter-949589df5-xqwz4       0/1     ContainerCreating   0           31s
mt-broker-ingress-6ccb8d9578-bjfrb     0/1     ContainerCreating   0           31s

C:\Users\联想\istio-1.29.2>kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istio-egressgateway-5999478b6d-pg24f 1/1     Running   0           18m
istio-ingressgateway-584467857d-fkpbr 1/1     Running   0           18m
istiod-6c4f4b85b7-6llth              1/1     Running   0           19m

C:\Users\联想\istio-1.29.2>

```

The following pods are all in 1/1 Running status:

activator - Manages network traffic and activates idle services
 autoscaler - Automatically scales pods based on traffic
 controller - Manages Knative Serving resources
 net-istio-controller - Istio networking integration
 net-istio-webhook - Webhook for Istio networking validation
 webhook - Admission webhook for resource validation

3.2 Installing Knative Eventing

Knative Eventing was installed using the eventing YAML manifest. This component enables event-driven architecture with brokers, triggers, and sources.

```
C:\Users\联想\istio-1.29.2>kubectl get pods -n knative-serving
NAME                                READY    STATUS    RESTARTS   AGE
activator-595d9dccc8-8mmbw          1/1      Running   0           8m29s
autoscaler-6bffb656cd-hw5v4         1/1      Running   0           8m28s
controller-6b75568fc-k49gf          1/1      Running   0           8m28s
net-istio-controller-5747656bdd-kd5rl 1/1      Running   0           6m20s
net-istio-webhook-645496f8b9-24nxc   1/1      Running   0           6m20s
webhook-76bcc8bb77-99qhd            1/1      Running   0           8m28s

C:\Users\联想\istio-1.29.2>kubectl get pods -n knative-eventing
NAME                                READY    STATUS              RESTARTS   AGE
eventing-controller-86f855d85-mx6v7 0/1      ImagePullBackOff    0           35s
eventing-webhook-74765bc895-9bm8j    0/1      ContainerCreating   0           34s
imc-controller-fbd87f5ff-7gv7t       0/1      ContainerCreating   0           33s
imc-dispatcher-5c6c6ddd98-cgzld      0/1      ContainerCreating   0           33s
job-sink-7c69f84bb5-mbvsl            1/1      Running             0           35s
mt-broker-controller-55f7cdbcc7-xrbc9 0/1      ContainerCreating   0           31s
mt-broker-filter-949589df5-xqwez4    0/1      ContainerCreating   0           31s
mt-broker-ingress-6ccb8d9578-bjfrb    0/1      ContainerCreating   0           31s

C:\Users\联想\istio-1.29.2>kubectl get pods -n istio-system
NAME                                READY    STATUS    RESTARTS   AGE
istio-egressgateway-5999478b6d-pg24f 1/1      Running   0           18m
istio-ingressgateway-584467857d-fkpbr 1/1      Running   0           18m
istiod-6c4f4b85b7-6llth              1/1      Running   0           19m

C:\Users\联想\istio-1.29.2>
```

The output shows various Knative Eventing components being created:

eventing-controller - ImagePullBackOff (temporary, resolved later)

eventing-webhook - ContainerCreating

imc-controller - In-Memory Channel controller

imc-dispatcher - In-Memory Channel dispatcher

job-sink - Running

mt-broker-controller - Multi-tenant broker controller

mt-broker-filter - Broker filter

mt-broker-ingress - Broker ingress

Note: The ImagePullBackOff status for eventing-controller was temporary and resolved automatically after a few minutes as Kubernetes retried pulling the image.

3.3 Verifying Istio Components

Knative requires a networking layer. Istio was already installed from Lab 4, and the net-istio.yaml was applied to integrate Knative with Istio.

All Istio components are running:

istio-egressgateway - 1/1 Running

istio-ingressgateway - 1/1 Running

istiod - 1/1 Running

3.4 Creating a Knative Service (Hello World)

A Knative Service was created using a YAML configuration file (hello-service.yaml).

The service defines a serverless application that scales to zero when idle.

```
C:\Users\联想\istio-1.29.2>kubectl apply -f hello-service.yaml
Warning: Kubernetes default value is insecure, Knative may default this to secure in a future release: spec.template.spec.containers[0].securityContext.allowPrivilegeEscalation, spec.template.spec.containers[0].securityContext.capabilities, spec.template.spec.containers[0].securityContext.runAsNonRoot, spec.template.spec.containers[0].securityContext.seccompProfile
service.serving.knative.dev/hello created

C:\Users\联想\istio-1.29.2>kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
hello-00001-deployment-5967d56b7-qjcbj 0/2     ContainerCreating   0           7s
hello-00001-deployment-5967d56b7-qjcbj 1/2     Running             0           4m21s
hello-00001-deployment-5967d56b7-qjcbj 2/2     Running             0           4m22s

C:\Users\联想\istio-1.29.2>kubectl get ksvc
NAME    URL                                LATESTCREATED   LATESTREADY   READY   REASON
hello   http://hello.default.svc.cluster.local hello-00001      hello-00001   True
```

The command `kubectl apply -f hello-service.yaml` created the service. The pod went through stages:

0/2 ContainerCreating - Both containers (user container + sidecar) being created

1/2 Running - User container running, sidecar starting

2/2 Running - Both containers fully operational

3.5 Verifying Knative Service Status

After deployment, the Knative service status was checked using `kubectl get ksvc`.

```
Profile
service.serving.knative.dev/hello created

C:\Users\联想\istio-1.29.2>kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
hello-00001-deployment-5967d56b7-qjcbj 0/2     ContainerCreating   0           7s
hello-00001-deployment-5967d56b7-qjcbj 1/2     Running             0           4m21s
hello-00001-deployment-5967d56b7-qjcbj 2/2     Running             0           4m22s

C:\Users\联想\istio-1.29.2>kubectl get ksvc
NAME    URL                                LATESTCREATED   LATESTREADY   READY   REASON
hello   http://hello.default.svc.cluster.local hello-00001      hello-00001   True

C:\Users\联想\istio-1.29.2>kubectl get svc istio-ingressgateway -n istio-system
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
istio-ingressgateway               LoadBalancer        10.106.70.111   <pending>        15021:30550/TCP,80:31066/TCP,443:32367/TCP,31400:30443/TCP,15443:31993/TCP 40m

C:\Users\联想\istio-1.29.2>kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
```

The output shows:

NAME: hello

URL: `http://hello.default.svc.cluster.local` (internal cluster URL)

LATESTCREATED: hello-00001

LATESTREADY: hello-00001

READY: True

3.6 Accessing the Knative Service via Istio Ingress

To access the Knative service externally, port forwarding was set up to the Istio ingress gateway.

```
Profile
service.serving.knative.dev/hello created

C:\Users\联想\istio-1.29.2>kubectl get pods -w
NAME                                READY    STATUS             RESTARTS   AGE
hello-00001-deployment-5967d56b7-qjcbj 0/2      ContainerCreating   0           7s
hello-00001-deployment-5967d56b7-qjcbj 1/2      Running             0          4m21s
hello-00001-deployment-5967d56b7-qjcbj 2/2      Running             0          4m22s

C:\Users\联想\istio-1.29.2>kubectl get ksvc
NAME    URL                                LATESTCREATED    LATESTREADY    READY    REASON
hello   http://hello.default.svc.cluster.local hello-00001        hello-00001    True

C:\Users\联想\istio-1.29.2>kubectl get svc istio-ingressgateway -n istio-system
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
istio-ingressgateway               LoadBalancer 10.106.70.111  <pending>      15021:30550/TCP,80:31066/TCP,443:32367/TCP,31400:30443/TCP,15443:31993/TCP 40m

C:\Users\联想\istio-1.29.2>kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
```

The commands executed:

```
bash
```

```
kubectl get svc istio-ingressgateway -n istio-system
```

```
kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80
```

The ingress gateway shows EXTERNAL-IP: <pending> because Docker Desktop does not provide an external load balancer. Port forwarding provides a workaround for local access.

The Knative service can then be accessed at <http://localhost:8080> (with appropriate Host header or using the Magic DNS setup).

3.7 Knative Functions Setup

The func CLI was used to create a simple Go function. The command `func create -l go hello` creates a function skeleton.

3.8 Knative Eventing - PingSource and Event Display

Knative Eventing was tested by creating a PingSource (generates periodic events) and an Event Display service that logs received events.

```
C:\Users\联想\istio-1.29.2>kubectl apply -f trigger.yaml
trigger.eventing.knative.dev/display-trigger created

C:\Users\联想\istio-1.29.2>kubectl get pods -w
NAME                                READY    STATUS             RESTARTS   AGE
event-display-00001-deployment-84fd647d65-gkrqf 2/2      Running            0           42s
hello-00001-deployment-5967d56b7-qjcbj          2/2      Running            0           75m

C:\Users\联想\istio-1.29.2>kubectl logs -l app=event-display --tail=20
No resources found in default namespace.
```

The event-display service was deployed and shows 2/2 Running status, indicating the Knative serving container plus the sidecar proxy.

3.9 Creating the Trigger

A Trigger was created to connect the PingSource to the Event Display service. The trigger defines that events from the PingSource should be delivered to the event-display service.

```
C:\Users\联想\istio-1.29.2>kubectl apply -f trigger.yaml
trigger.eventing.knative.dev/display-trigger created

C:\Users\联想\istio-1.29.2>kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
event-display-00001-deployment-84fd647d65-gkrqf  2/2     Running   0           42s
hello-00001-deployment-5967d56b7-qjcbj          2/2     Running   0           75m

C:\Users\联想\istio-1.29.2>kubectl logs -l app=event-display --tail=20
No resources found in default namespace.
```

The trigger was successfully created:

text

trigger.eventing.knative.dev/display-trigger created

3.10 Verifying Event Reception

After allowing time for events to be generated (PingSource default: every minute), the logs of the event-display service were examined.

```
C:\Users\联想\istio-1.29.2>kubectl logs -l serving.knative.dev/service=event-display --tail=20
specversion: 1.0
type: dev.knative.sources.ping
source: /apis/v1/namespaces/default/pingsources/test-ping
id: e2a27a33-51d6-4653-b9e1-e8635a3798e8
time: 2026-05-11T10:31:00.480630923Z
Extensions,
knativearrivaltime: 2026-05-11T10:31:00.484523002Z
Data,
{"message": "Hello Knative Eventing!"}
c: cloudevents.Event
Context Attributes,
specversion: 1.0
type: dev.knative.sources.ping
source: /apis/v1/namespaces/default/pingsources/test-ping
id: 06f4dfcf-8317-4fa4-ae4a-b95e41317654
time: 2026-05-11T10:32:00.322046568Z
Extensions,
knativearrivaltime: 2026-05-11T10:32:00.32486428Z
Data,
{"message": "Hello Knative Eventing!"}
```

The logs show CloudEvents received with the following structure:

text

Context Attributes:

specversion: 1.0

type: dev.knative.sources.ping

source: /apis/v1/namespaces/default/pingsources/test-ping

id: e2a27a33-51d6-4653-b9e1-e8635a3798e8

time: 2026-05-11T10:31:00.480630923Z

Extensions:

knativearrivaltime: 2026-05-11T10:31:00.484523002Z

Data:

{"message": "Hello Knative Eventing!"}

This confirms that:

The PingSource is generating CloudEvents every minute

The Trigger is correctly routing events to the event-display service

The event-display service is successfully receiving and logging events

3.11 Troubleshooting Event Display Logs

During the experiment, an attempt was made to view logs using `kubectl logs -l`

app=event-display. This returned "No resources found" because the label selector app=event-display does not match the actual pod labels.

```
C:\Users\联想\istio-1.29.2>echo [2] 事件日志内容 (最近30行)
[2] 事件日志内容 (最近30行)

C:\Users\联想\istio-1.29.2>kubectl logs -l app=event-display --tail=30
No resources found in default namespace.

C:\Users\联想\istio-1.29.2>echo.

C:\Users\联想\istio-1.29.2>echo [3] 如果有错误, 查看 Pod 详情
[3] 如果有错误, 查看 Pod 详情

C:\Users\联想\istio-1.29.2>kubectl describe pod -l app=event-display -f ./data -i --namespace="knative"
```

The correct way to view logs is using:

bash

kubectl logs -l serving.knative.dev/service=event-display --tail=30

4. Issues and Solutions

Several issues were encountered during the experiment, along with their respective solutions

Issue	Cause	Solution
ImagePullBackOff for eventing-controller	Container image pull delay or transient network issue	Waited for automatic retry; Kubernetes successfully pulled the image after ~2 minutes
func.exe move access denied	Insufficient permissions to move to C:\Windows\System32\	Used func.exe from current directory by adding to PATH environment variable or using full path
kubectl logs -l app=event-display returns no resources	Incorrect label selector; Knative services use different labels	Used correct label serving.knative.dev/service=event-display instead of app=event-display
EXTERNAL-IP pending for istio-ingressgateway	Docker Desktop does not provision external load balancers	Used kubectl port-forward as a workaround for local access
Warning about securityContext defaults	Knative may default secure options in future releases	Warning only; can be ignored for this lab, but production deployments should specify securityContext
Issue	Cause	Solution

5. Key Concepts Summary

The core Kubernetes operations covered in this lab include:

Operation	Command	Description
Start Minikube for Knative	<code>minikube start --memory=15360 --cpus=8</code>	Allocates sufficient resources for Knative
Install Knative Serving	<code>kubectl apply -f serving-crds.yaml</code> then <code>serving-core.yaml</code>	Installs Serving CRDs and controllers
Install Knative Eventing	<code>kubectl apply -f eventing.yaml</code>	Installs Eventing components
Install net-istio	<code>kubectl apply -f net-istio.yaml</code>	Integrates Istio with Knative
Create Knative Service	<code>kubectl apply -f hello-service.yaml</code>	Deploys a serverless application
List Knative Services	<code>kubectl get ksvc</code>	Shows all services and their readiness
Check pods with sidecar	<code>kubectl get pods -w</code>	Monitors pod status (0/2 → 2/2 indicates sidecar injection)
Port forward to gateway	<code>kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80</code>	Provides external access locally
Create function	<code>func create -l go hello</code>	Scaffolds a new serverless function
Create Trigger	<code>kubectl apply -f trigger.yaml</code>	Connects event source to sink
View event logs	<code>kubectl logs -l serving.knative.dev/service=event-display</code>	Verifies event delivery

Key Kubernetes Concepts Explained:

Knative Serving: Component that manages serverless workloads with automatic scaling (including scale-to-zero).

Knative Eventing: Component that enables event-driven architecture with brokers, triggers, and sources.

Knative Service (ksvc): A resource that manages the full lifecycle of a serverless application including revisions and routing.

Sidecar Injection: Knative automatically injects a queue-proxy sidecar container to handle traffic management and metrics.

PingSource: A built-in event source that generates CloudEvents on a cron schedule.

Trigger: Defines a subscription from an event source/broker to a target service.

CloudEvents: A specification for describing event data in a common way.

Activator: Knative component that holds requests for idle services until pods are started.

Autoscaler: Monitors request metrics and scales pods up/down based on demand.

6. Conclusion

This lab successfully demonstrated the complete installation and operation of the Knative serverless platform on a local Kubernetes cluster. The following key achievements were accomplished:

Minikube Configuration: A Minikube cluster with 15GB memory and 8 CPU cores was configured specifically for Knative's resource requirements.

Knative Serving Installation: The Serving component was successfully installed with all controllers (activator, autoscaler, controller, webhooks) running in the knative-serving namespace.

Knative Eventing Installation: The Eventing component was installed and eventually all pods became healthy after initial image pull delays.

Istio Integration: The net-istio controller was installed, providing networking and ingress capabilities for Knative services.

Serverless Service Deployment: A hello-world Knative Service was deployed, demonstrating automatic sidecar injection (2/2 pods) and the ksvc custom resource.

Event-Driven Architecture: Knative Eventing was tested with:

PingSource generating periodic CloudEvents

Trigger routing events to the target service

Event Display service successfully receiving and logging events

Observability: Event logs confirmed that CloudEvents were properly delivered with correct attributes (specversion, type, source, id, time, and data).